

Overview

Part 13 —  AI Security · 13-ai-security/README.md



Module 13 — AI Security

Where you are: Stop 13 of 16 on the end-to-end AI track. **Time:** ~18–22 hours · **Prereq:** Module 04 [04-llm] (how LLMs behave), Module 06 [06-ai-agents] (tool-using agents) and Module 09 [09-mcp] (MCP) — you cannot reason about attack surface on a system you don't understand the shape of.

Every earlier module built capability: a model that follows instructions, an agent that can act, a protocol that lets it reach your tools and data. This module is about the fact that "follows instructions" and "can act" are also, structurally, an attack surface — and that the attacker's instructions arrive through the exact same channel as the user's. AI security is not "the OWASP Top 10 with an LLM sticker on it": prompt injection has no clean fix the way SQL injection does, because there is no prepared-statement equivalent that separates instructions from data inside a language model's context window. This module teaches you to recognize the failure class, build layered defenses that reduce risk without pretending to eliminate it, and reason like a red-teamer about your own system before someone else does.

Learning objectives

By the end of this module you can:

1. Name and explain, at a working level, each category in the current OWASP Top 10 for LLM Applications, and map an incident report or a CVE description to the right category.
2. Explain *why* prompt injection is structurally hard to solve — not just describe direct vs. indirect injection — and design a defense-in-depth stack that meaningfully reduces (not eliminates) the risk.
3. Spot the "lethal trifecta" (untrusted input + sensitive data access + external communication) in an agent architecture diagram, and redesign the architecture to remove it.
4. Recognize the major jailbreak technique families well enough to test for them, without needing a cookbook of working exploits.
5. Identify the AI-specific data risks in a system — training data poisoning, memorization leakage, RAG over-retrieval of sensitive documents, embedding inversion — and name a control for each.
6. Describe what a structured LLM red-team engagement produces that a traditional pentest report does not.
7. Place guardrails (input/output classifiers, PII redaction, rate limiting) correctly in a request-flow diagram, and explain what each one does and does not catch.

Suggested path

#	Do this	File	Time
1	OWASP frame, prompt injection, jailbreaking	notes/01-owasp-and-prompt-injection.md	4h
2	Agentic security, data risks, red-teaming, guardrails, supply chain	notes/02-agentic-security-and-red-teaming.md	4h
3	Watch a pipeline fall for an injection, then defend it layer by layer	code/prompt_injection_defense_demo.py	2h
4	Statically flag the lethal trifecta in five agent configs	code/lethal_trifecta_analyzer.py	1.5h
5	Slides	slides/	1h
6	Lab	lab/EXERCISES.md	5h
7	Papers & primary sources	papers/PAPERS.md	4h

The 18 terms you must own

prompt injection (direct/indirect) · lethal trifecta · privilege separation · jailbreak · many-shot jailbreaking · crescendo attack · insecure output handling · training data poisoning · membership inference · embedding inversion · model extraction · excessive agency · confused deputy · human-in-the-loop gate · red-teaming · guardrail · PII redaction · AI supply chain (SBOM for models)

Exit check

You can take an architecture diagram for an agent you did not design, mark every place the lethal trifecta appears, propose a specific fix for each (split agent, remove a capability, or add an approval gate), and hand a colleague a one-page threat model that names which OWASP LLM category each risk falls under and which layer of your defense-in-depth stack catches it — including an honest statement of what still isn't caught.